



UNIMINUTO
Corporación Universitaria Minuto de Dios
Educación de calidad al alcance de todos
Vigilada MinEducación



CORPORACIÓN UNIVERSITARIA MINUTO DE DIOS



Autores

Miguel Angel Ahumada Duarte

ID 701522

Juan Sebastian Lara Suarez

ID 715655

Presentado a

JAIDER OSPINA NAVAS

NRC-75062

Bogotá D.C.
Abril de 2025

2. Introducción

2.1 Contexto del trabajo

El presente informe técnico documenta el diseño, implementación y aseguramiento de SecureVault, una aplicación contenerizada de libre uso orientada a la gestión segura de credenciales y secretos para equipos de desarrollo. El proyecto se enmarca en un enfoque DevSecOps, donde la seguridad no se incorpora como una actividad posterior al desarrollo, sino como una práctica transversal integrada desde la planeación, el código, la construcción de imágenes, las pruebas, la publicación y el despliegue.

SecureVault fue desarrollado como una plataforma local, contenerizada y reproducible, capaz de almacenar, consultar, rotar y auditar secretos como contraseñas, tokens, API keys, certificados u otros valores sensibles. La aplicación se diseñó bajo una arquitectura basada en servicios independientes, usando comunicación HTTP/REST para los flujos sincrónicos y mensajería AMQP mediante RabbitMQ para procesos asíncronos.

El propósito principal del proyecto no es únicamente demostrar una aplicación funcional, sino evidenciar un ciclo completo de ingeniería DevSecOps: arquitectura de microservicios, contenerización segura, automatización CI/CD, escaneo de código, escaneo de dependencias, análisis de imágenes Docker, pruebas unitarias, DAST, IaC scanning, despliegue automatizado y documentación técnica.

2.2 Justificación de la aplicación elegida

La gestión insegura de secretos es uno de los riesgos más frecuentes en entornos de desarrollo y operación. Es común encontrar credenciales embebidas en código fuente, archivos .env publicados accidentalmente, tokens sin rotación, claves compartidas sin trazabilidad o ausencia de auditoría sobre el acceso a valores sensibles. Estos problemas pueden derivar en exposición de información, movimientos laterales, escalamiento de privilegios y compromiso de servicios internos.

SecureVault fue elegido como caso de uso porque permite conectar de forma natural el desarrollo de software con controles reales de ciberseguridad. La aplicación permite evidenciar prácticas como:

- Autenticación basada en JWT para proteger endpoints.
- Control de acceso basado en roles: administrador, editor y visualizador.
- Cifrado de secretos a nivel de aplicación antes de persistirlos en PostgreSQL.
- Registro de auditoría sobre operaciones sensibles.
- Rotación y expiración de secretos como control operativo.
- Procesamiento asíncrono mediante worker y RabbitMQ.
- Escaneo continuo de código, dependencias, imágenes e infraestructura.
- Despliegue reproducible mediante Docker Compose e infraestructura como código.

A diferencia de una aplicación meramente demostrativa, SecureVault representa un problema real de seguridad: cómo administrar secretos internos con trazabilidad y controles mínimos de acceso. Esta elección permite que el pipeline DevSecOps tenga sentido práctico, ya que el propio dominio de la aplicación exige controles de seguridad desde el diseño.

2.3 Objetivo general

Diseñar e implementar un pipeline DevSecOps de ciclo completo para SecureVault, una aplicación contenerizada de gestión segura de secretos, integrando controles automatizados de seguridad, pruebas, construcción de imágenes, análisis de vulnerabilidades, despliegue e infraestructura como código.

2.4 Objetivos específicos

1. Implementar una aplicación basada en servicios contenerizados que incluya frontend SPA, API backend, worker asíncrono, base de datos y broker de mensajería.
2. Incorporar autenticación JWT y control básico de roles para proteger operaciones sensibles sobre secretos.
3. Cifrar los valores secretos antes de almacenarlos en la base de datos.
4. Registrar eventos de auditoría asociados a creación, consulta, rotación, actualización y eliminación de secretos.
5. Automatizar el análisis de seguridad del código fuente mediante herramientas SAST y detección de secretos.
6. Automatizar el análisis de dependencias y vulnerabilidades de imágenes Docker mediante Trivy.
7. Ejecutar pruebas unitarias de backend y frontend dentro del pipeline CI/CD.
8. Ejecutar análisis dinámico de seguridad mediante OWASP ZAP contra un entorno de staging levantado por Docker Compose.
9. Integrar revisión de infraestructura como código mediante Checkov.
10. Publicar imágenes Docker versionadas y preparar un flujo de despliegue automatizado con Ansible.
11. Documentar el modelo de amenazas, las decisiones de arquitectura, los riesgos residuales y las lecciones aprendidas.

2.5 Alcance del proyecto

El alcance implementado corresponde a un MVP funcional con orientación DevSecOps, compuesto por:

- Frontend SPA en React + Vite + Tailwind CSS.
- Backend API / API Gateway en FastAPI.
- Worker asíncrono en Python + Pika.
- Broker RabbitMQ.
- Base de datos PostgreSQL.
- Autenticación JWT con access token y refresh token.

- Roles: admin, editor, viewer.
- Cifrado de valores secretos mediante Fernet.
- Auditoría persistente en PostgreSQL.
- Dockerfiles por servicio.
- Docker Compose como orquestación principal local / producción simulada.
- Pipeline GitHub Actions con Gitleaks, Bandit, Semgrep, Trivy, Pytest, Jest, OWASP ZAP, Checkov y publicación en Docker Hub.
- IaC mediante Terraform y Ansible.
- Manifiestos Kubernetes como referencia de orquestación.
- Stack de monitoreo opcional con Prometheus, Grafana y Loki.

2.6 Información técnica confirmada del proyecto

La siguiente información fue validada sobre la estructura y archivos del proyecto:

Elemento	Confirmación
Backend	Existe api-gateway/ con FastAPI, routers auth.py, secrets.py, audit.py, modelos SQLAlchemy y pruebas Pytest.
Frontend	Existe frontend/ con React, Vite, Tailwind, páginas de login, registro, dashboard, secretos, auditoría y cambio de contraseña.
Worker	Existe worker-audit/worker.py, implementado en Python con Pika y SQLAlchemy.
Base de datos	PostgreSQL 16 mediante contenedor postgres:16-alpine.
Broker	RabbitMQ rabbitmq:3.13-management-alpine con colas secret_events y secret_alerts.
Docker Compose	docker-compose.yml levanta PostgreSQL, RabbitMQ, API, worker y frontend.
Pipeline CI/CD	.github/workflows/ci.yml contiene Gitleaks, Bandit, Semgrep, Trivy, Pytest, Jest, ZAP, Checkov y publicación de imágenes.
Despliegue	.github/workflows/deploy.yml ejecuta Ansible si el pipeline CI termina exitosamente en main.
IaC	Existen archivos Terraform y Ansible en infrastructure/.

Elemento	Confirmación
Orquestación	Existen manifiestos Kubernetes en orquestacion/.
Documentación	Existen manuales base en docs/.
Licencia	Existe LICENSE con licencia MIT.

3. Arquitectura

3.1 Visión general

SecureVault se diseñó con una arquitectura distribuida basada en contenedores. Cada componente ejecuta una responsabilidad específica y se comunica mediante protocolos claros:

- HTTP/REST para comunicación entre frontend y backend.
- SQLAlchemy/PostgreSQL para persistencia transaccional.
- AMQP/RabbitMQ para eventos asíncronos.
- Docker Network sv_network para aislamiento lógico entre servicios.
- Docker volumes para persistencia de PostgreSQL.

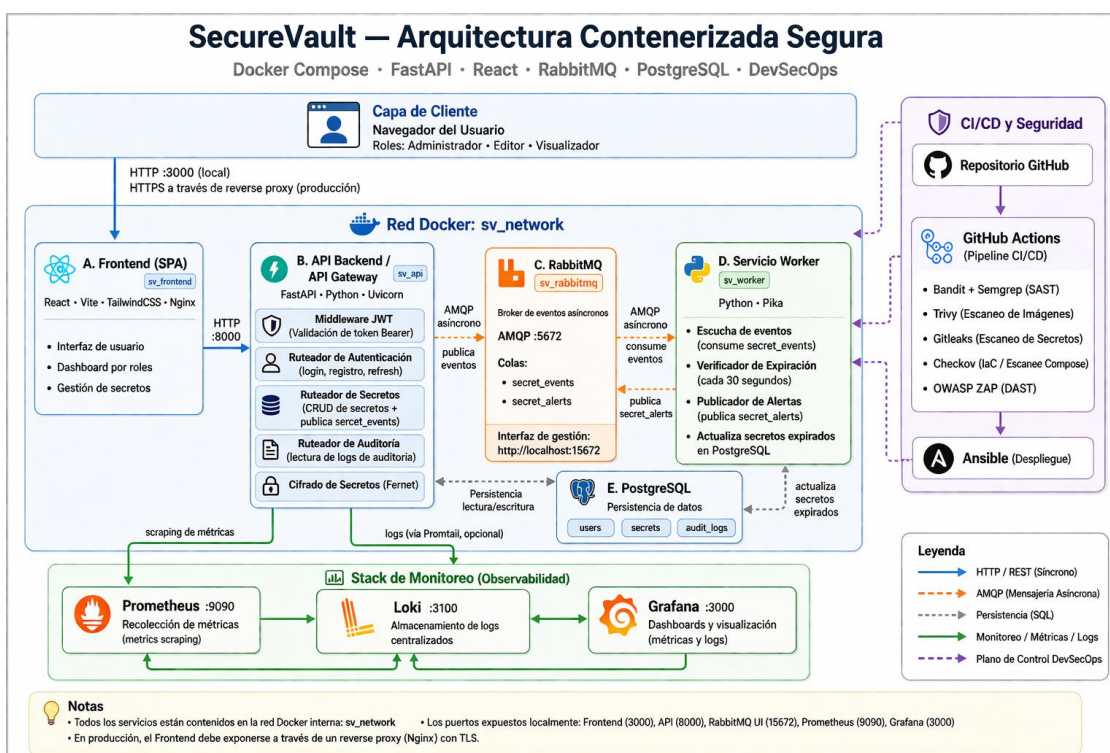
La arquitectura se compone de los siguientes servicios principales:

Componente	Tecnología	Responsabilidad
Frontend SPA	React + Vite + Tailwind + Nginx	Interfaz web para usuarios administradores, editores y visualizadores.
Backend API / API Gateway	FastAPI + Uvicorn	Autenticación, autorización, gestión de secretos, auditoría y publicación de eventos.
Worker Audit	Python + Pika	Consumo de eventos y verificación periódica de expiración de secretos.
Message Broker	RabbitMQ	Comunicación asíncrona mediante colas secret_events y secret_alerts.
Persistencia	PostgreSQL + SQLAlchemy	Almacenamiento de usuarios, secretos cifrados, accesos y auditoría.
CI/CD	GitHub Actions	Automatización de escaneos, pruebas,

Componente	Tecnología	Responsabilidad
		construcción, publicación y despliegue.
IaC/Deploy	Terraform + Ansible	Definición y automatización de infraestructura/despliegue.
Monitoreo opcional	Prometheus + Grafana + Loki	Recolección de métricas, logs y visualización.

3.2 Diagrama de arquitectura containerizada segura

Figura 1. SecureVault — Arquitectura Containerizada Segura.



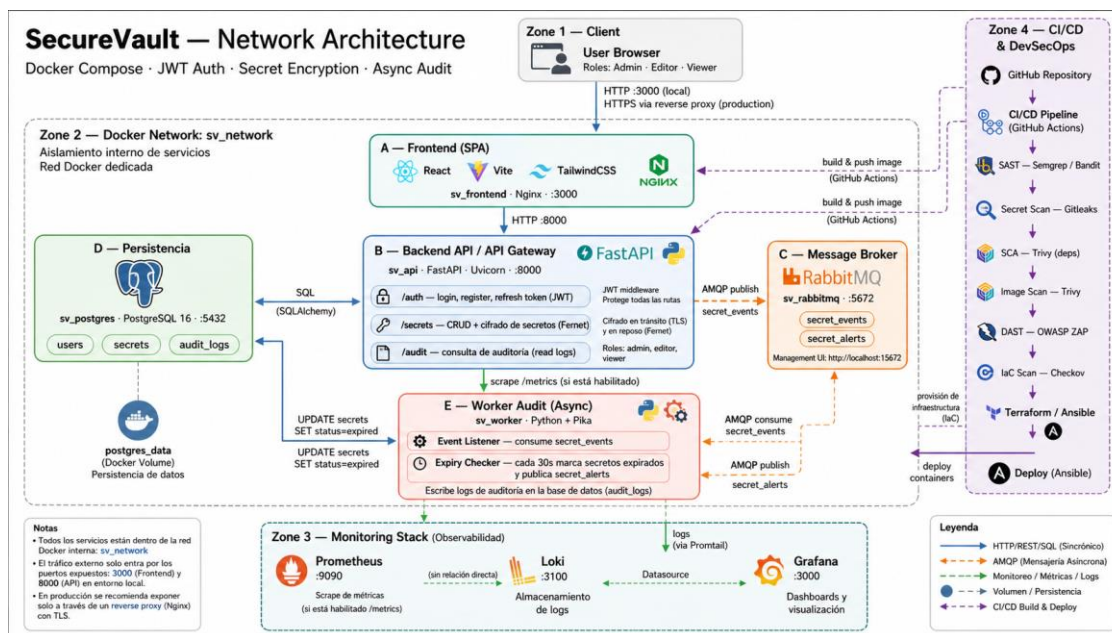
SecureVault — Arquitectura Containerizada Segura

Descripción de la figura:

El diagrama muestra el flujo entre la capa de cliente, la red interna Docker sv_network, los servicios containerizados y el plano de control DevSecOps. El usuario accede al frontend por HTTP :3000 en entorno local. En producción se recomienda exposición mediante reverse proxy con TLS. El frontend consume la API FastAPI por HTTP :8000. La API gestiona autenticación, autorización, cifrado de secretos y auditoría. La comunicación asíncrona con RabbitMQ permite desacoplar el procesamiento del worker. PostgreSQL mantiene la persistencia de datos.

3.3 Diagrama de arquitectura de red

Figura 2. SecureVault — Network Architecture.



SecureVault — Network Architecture

Descripción de la figura:

El diagrama de red separa el sistema por zonas:

- Zona 1 — Cliente: navegador del usuario con roles administrador, editor y visualizador.
- Zona 2 — Red Docker sv_network: frontend, backend, RabbitMQ, PostgreSQL y worker.
- Zona 3 — Monitoreo: Prometheus, Loki y Grafana como extensión opcional de observabilidad.
- Zona 4 — CI/CD & DevSecOps: GitHub Repository, GitHub Actions, SAST, secret scan, SCA, image scan, DAST, IaC scan, Terraform y Ansible.

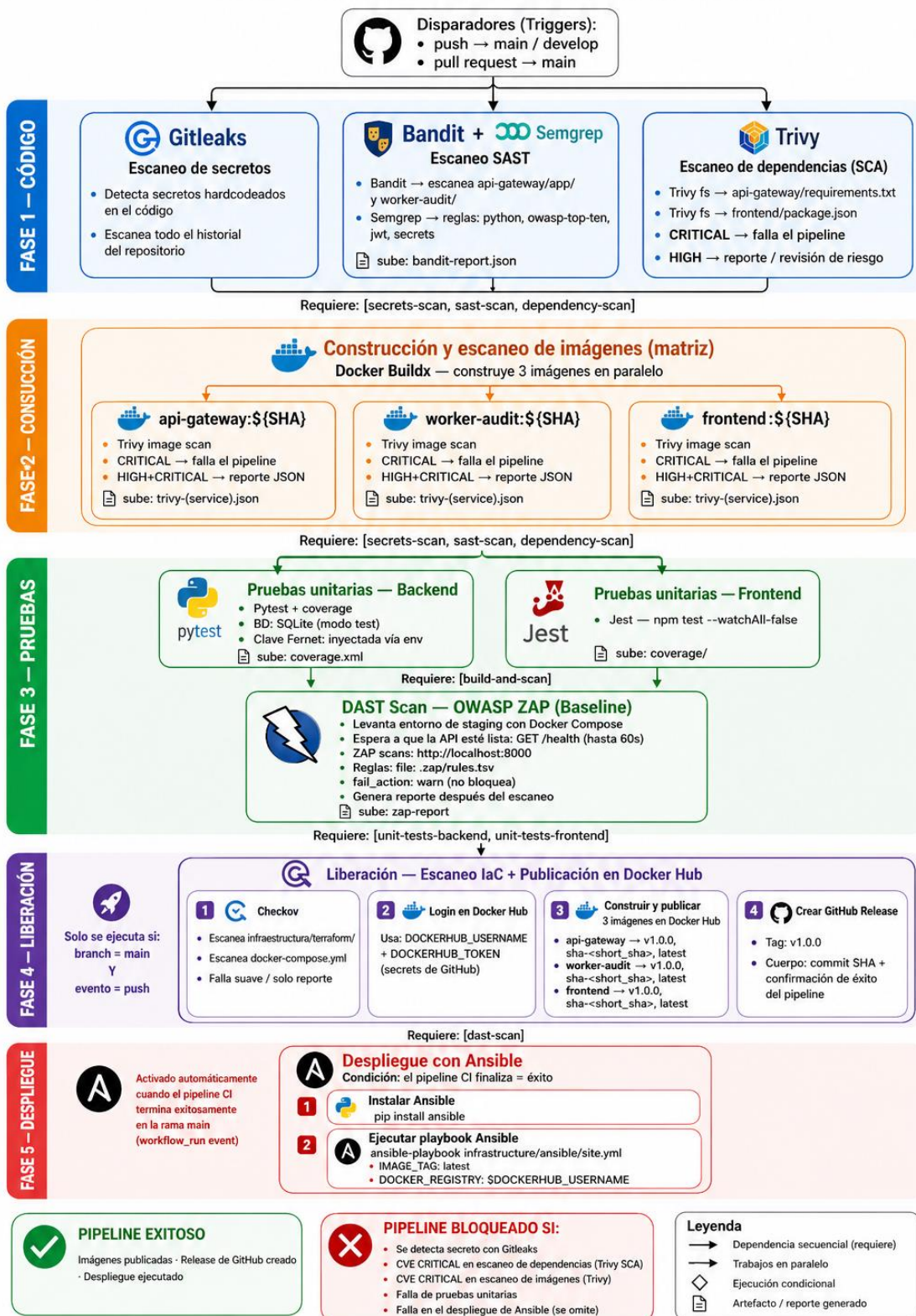
Esta separación permite explicar aislamiento, flujos de tráfico, persistencia y automatización de seguridad.

3.4 Diagrama del pipeline DevSecOps

Figura 3. SecureVault — Pipeline DevSecOps CI/CD.

SecureVault — Pipeline DevSecOps CI/CD

GitHub Actions · Activado con push a main / develop · PR a main



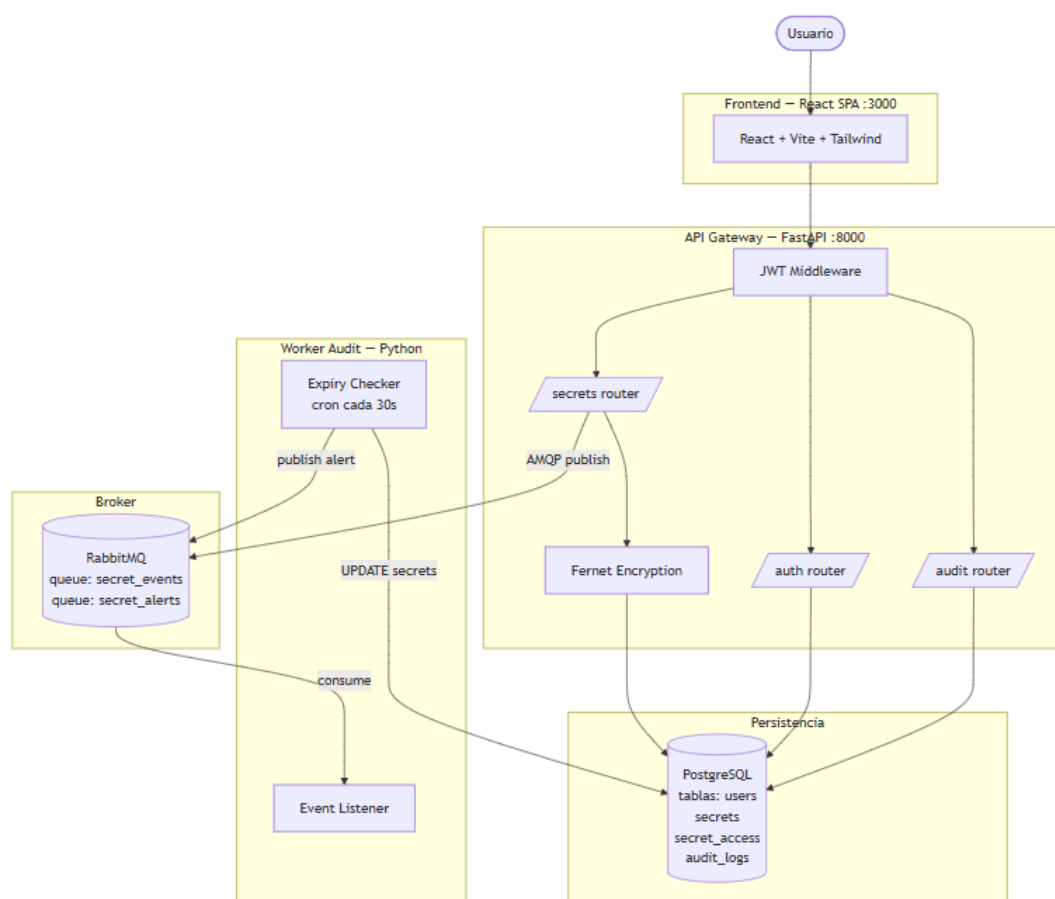
SecureVault — Pipeline DevSecOps CI/CD

Descripción de la figura:

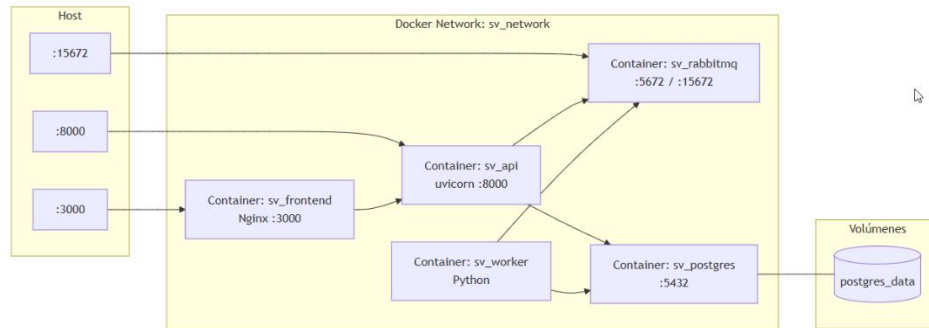
El pipeline se organiza en cinco fases:

1. Código: detección de secretos, SAST y análisis de dependencias.
2. Construcción: build paralelo de imágenes Docker y escaneo de CVEs.
3. Pruebas: pruebas unitarias backend/frontend y DAST con OWASP ZAP.
4. Liberación: Checkov, login Docker Hub, publicación de imágenes y creación de release.
5. Despliegue: ejecución automatizada con Ansible tras éxito del pipeline CI.

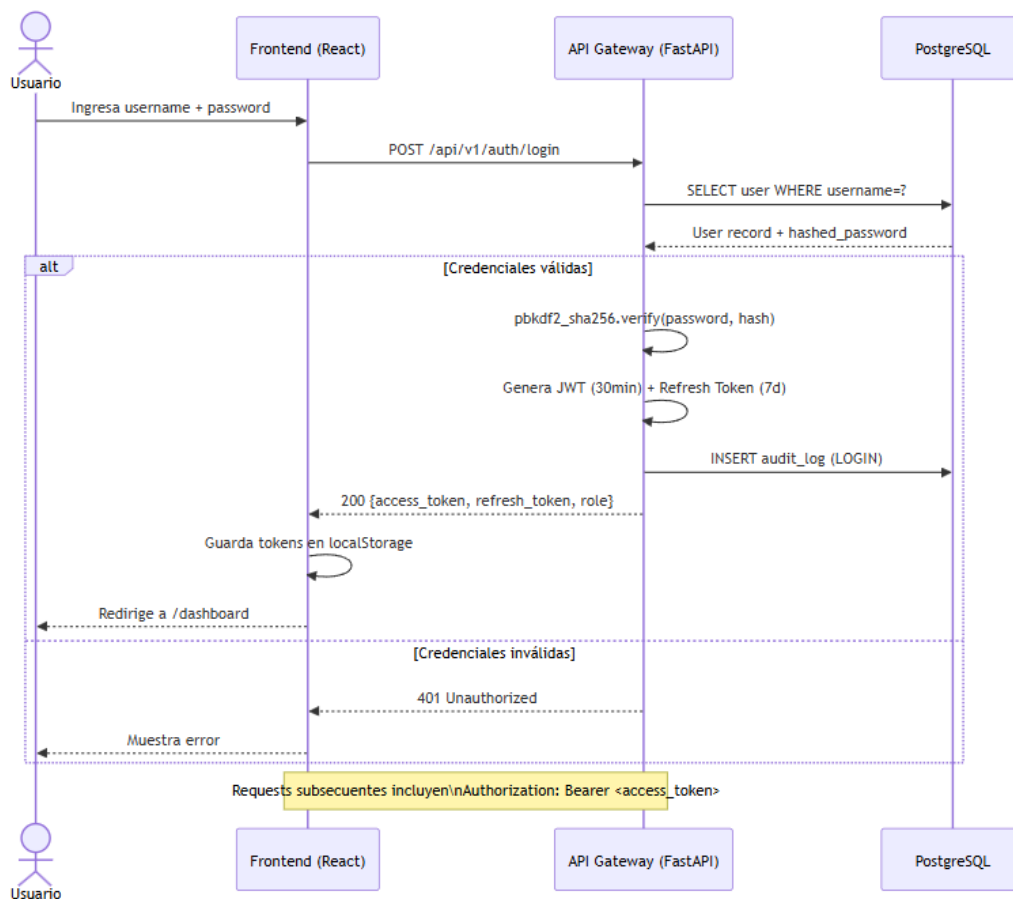
3.5 Diagrama de componentes UML



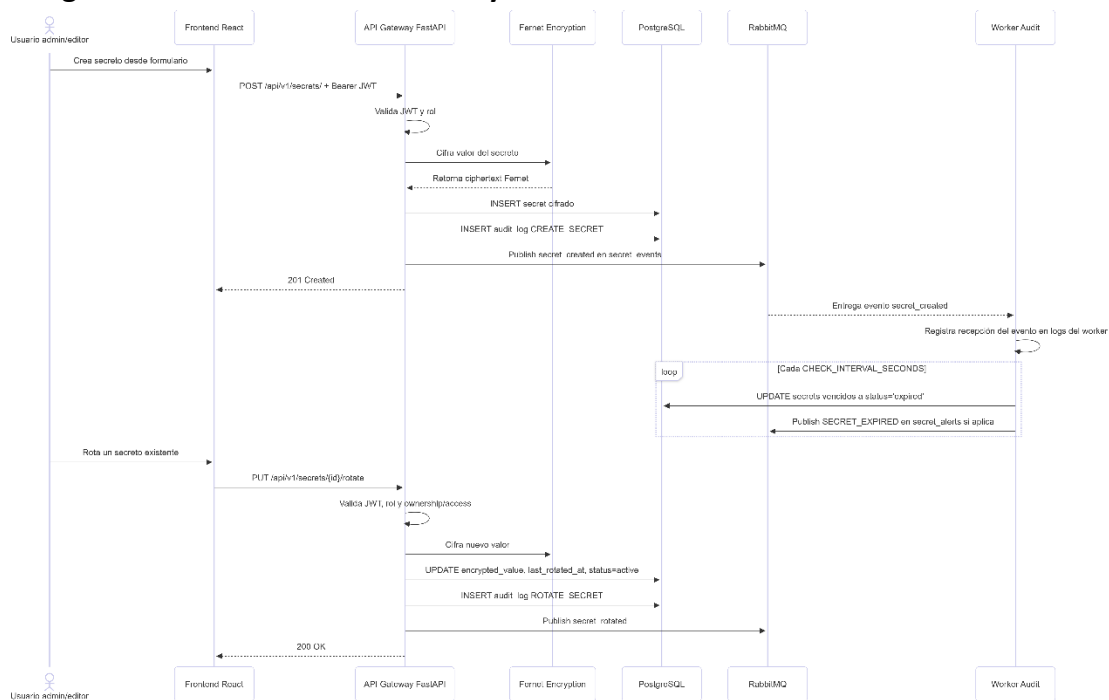
3.6 Diagrama de despliegue UML



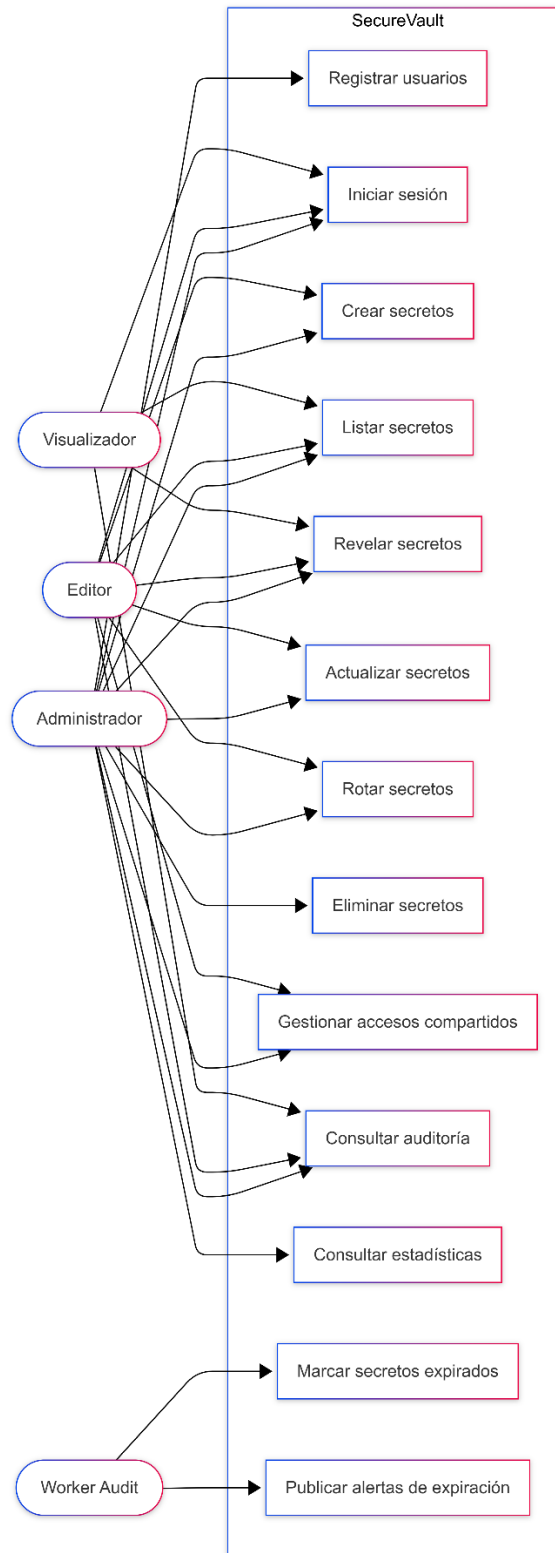
3.7 Diagrama de secuencia — autenticación



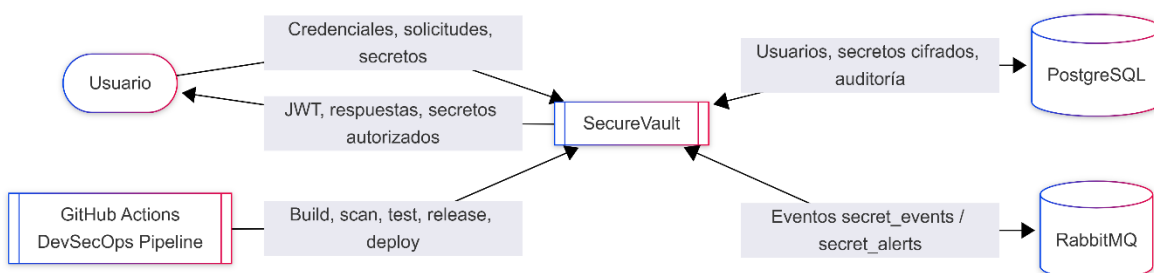
3.8 Diagrama de secuencia — creación y rotación de secretos



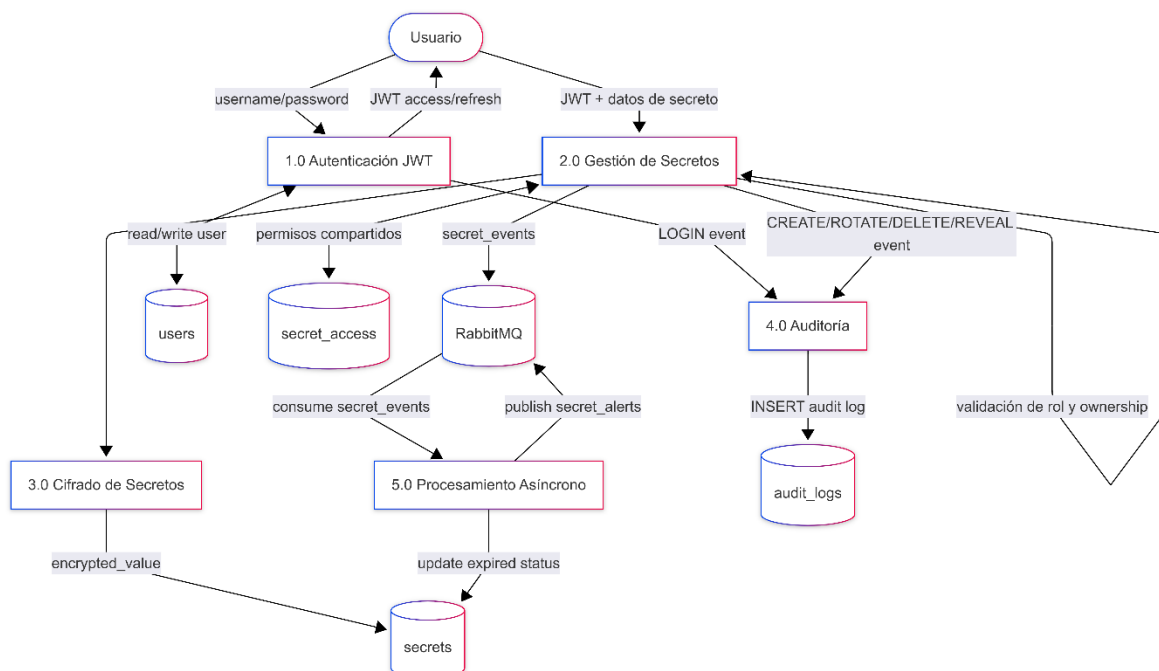
3.9 Diagrama de casos de uso



3.10 DFD nivel 0



3.11 DFD nivel 1



3.12 Decisiones de diseño

Decisión	Justificación
FastAPI para API Gateway	Permite construir una API REST ligera, documentada automáticamente con OpenAPI y adecuada para microservicios Python.
React + Vite para frontend	SPA liviana, rápida de compilar y sencilla de containerizar con Nginx.
PostgreSQL	Base de datos relacional robusta para usuarios, secretos, permisos y auditoría.

Decisión	Justificación
RabbitMQ	Broker AMQP adecuado para desacoplar eventos de secretos y procesamiento asíncrono.
Worker separado	Permite ejecutar verificación de expiración sin bloquear la API.
Fernet	Cifrado simétrico autenticado a nivel de aplicación para proteger el valor de los secretos antes de persistirlos.
Docker Compose	Orquestación reproducible para entorno local y producción simulada.
GitHub Actions	Automatización CI/CD integrada al repositorio.
Trivy, Gitleaks, Bandit, Semgrep, ZAP y Checkov	Herramientas FOSS o de libre uso alineadas con DevSecOps.

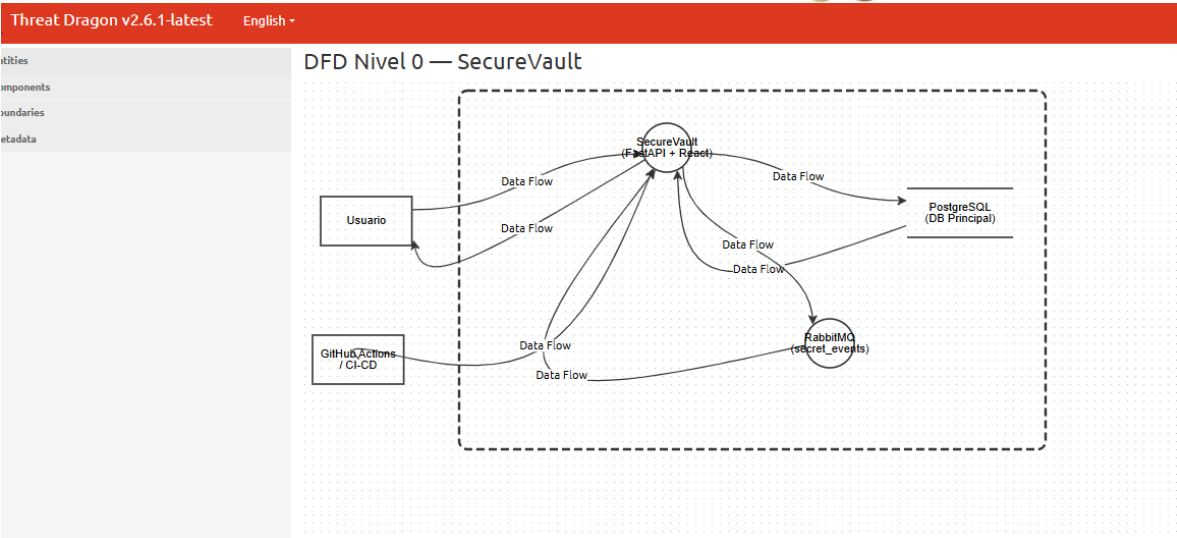
4. Modelado de Amenazas

4.1 Metodología

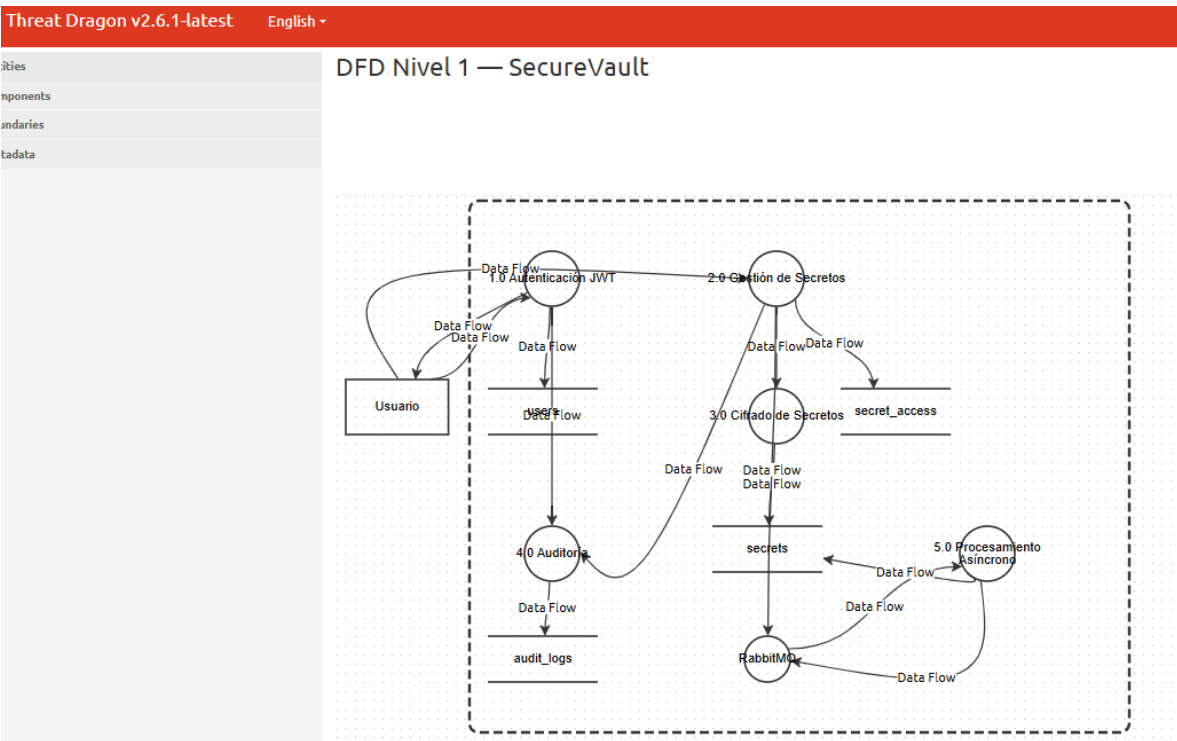
El modelado de amenazas se estructuró bajo el enfoque STRIDE, considerando los flujos principales del sistema: autenticación, almacenamiento de secretos, revelado de secretos, rotación, auditoría, mensajería asíncrona, persistencia y pipeline CI/CD.

El repositorio contiene un archivo `docs/architecture/threat-model.json`, preparado para documentar el modelo de amenazas con OWASP Threat Dragon. Para el informe final se debe adjuntar o capturar el modelo visual desde Threat Dragon, incluyendo DFD nivel 0 y DFD nivel 1.

OWASP Threat Dragon — DFD Nivel 0



OWASP Threat Dragon — DFD Nivel 1



4.2 Activos protegidos

Activo	Descripción	Criticidad
Credenciales de usuario	Usuario, correo, hash de contraseña y rol.	Alta
Secretos almacenados	API keys, contraseñas, tokens, certificados u otros valores sensibles.	Crítica
Clave Fernet	Clave simétrica usada para cifrar/descifrar secretos.	Crítica
SECRET_KEY JWT	Clave de firma de tokens JWT.	Crítica
Audit logs	Evidencia de operaciones sobre secretos y autenticación.	Alta
Base de datos PostgreSQL	Persistencia principal del sistema.	Alta
Colas RabbitMQ	Eventos secret_events y secret_alerts.	Media
Pipeline CI/CD	Canal de construcción, publicación y despliegue.	Alta
Docker Hub token	Credencial para publicación de imágenes.	Crítica

4.3 Límites de confianza

Límite	Descripción	Riesgo principal
Navegador ↔ Frontend	Usuario interactúa con la SPA.	XSS, robo de tokens, manipulación de requests.
Frontend ↔ API	Requests HTTP con Bearer token.	Token replay, acceso no autorizado.
API ↔ PostgreSQL	Persistencia de usuarios, secretos y auditoría.	SQL injection, fuga de datos, manipulación directa.
API ↔ RabbitMQ	Publicación de eventos asíncronos.	Eventos falsificados, pérdida de mensajes.
Worker ↔ PostgreSQL	Actualización de secretos expirados.	Manipulación de estados o abuso de permisos DB.

Límite	Descripción	Riesgo principal
GitHub Actions ↔ Docker Hub	Publicación de imágenes.	Compromiso de secrets CI/CD, publicación maliciosa.
Docker Host ↔ Contenedores	Red interna y puertos expuestos.	Exposición de servicios internos o malas configuraciones.

4.4 Amenazas STRIDE y contramedidas

Categoría STRIDE	Amenaza	Componente afectado	Severidad	Contramedida implementada	Estado
Spoofing	Suplantación de usuario mediante credenciales válidas robadas.	Auth/API	Alta	JWT firmado, expiración de token, refresh token y validación de usuario activo.	Mitigado parcialmente
Spoofing	Uso de token JWT manipulado.	API Gateway	Alta	Validación de firma y expiración con python-jose.	Mitigado
Spoofing	Intento de acceso a API sin Bearer token.	Endpoints protegidos	Alta	Dependencia a get_current_user con HTTPBearer.	Mitigado
Tampering	Modificación del valor de un secreto antes de persistirse.	Secrets Router	Alta	Cifrado Fernet del valor antes de guardarlo.	Mitigado

Categoría STRIDE	Amenaza	Componente afectado	Severidad	Contramedida implementada	Estado
Tampering	Manipulación directa de registros en PostgreSQL.	DB	Alta	Contenedores aislados en red Docker; credenciales por variables de entorno; acceso controlado desde servicios.	Mitigado parcialmente
Tampering	Alteración de mensajes AMQP.	RabbitMQ	Media	Colas durables y uso interno en red Docker.	Mitigado parcialmente
Repudiation	Usuario niega haber revelado, creado, rotado o eliminado un secreto.	API / Audit	Media	Registro en audit_logs desde la API para acciones sensibles.	Mitigado
Repudiation	Falta de trazabilidad de eventos asíncronos.	Worker	Media	Logs del worker y publicación de eventos de alerta.	Mitigado parcialmente
Information Disclosure	Exposición de secretos en texto plano en base de datos.	PostgreSQL	Crítica	Cifrado de valores secretos con Fernet antes de persistir.	Mitigado
Information Disclosure	Exposición accidental de secretos en repositorio.	GitHub	Crítica	Gitleaks en pipeline y pre-commit.	Mitigado

Categoría STRIDE	Amenaza	Componente afectado	Severidad	Contramedida implementada	Estado
Information Disclosure	Usuario visualiza secretos de otro usuario.	API	Alta	Validación de rol, ownership y tabla secret_access.	Mitigado
Information Disclosure	Robo de JWT ante XSS por almacenamiento en cliente.	Frontend	Alta	Riesgo residual documentado; mitigación parcial con expiración y validación backend.	Aceptado
Denial of Service	Abuso del endpoint de login o creación masiva de secretos.	API	Media	Docker healthchecks y separación de servicios; rate limiting pendiente.	Aceptado
Denial of Service	Fallo del broker impide procesamiento asíncrono.	RabbitMQ/Worker	Media	Healthchecks y retry de conexión en worker.	Mitigado parcialmente
Elevation of Privilege	Viewer intenta crear, editar, rotar o eliminar secretos.	API	Alta	Validación explícita del rol viewer con respuesta 403.	Mitigado
Elevation of Privilege	Editor intenta acceder a	API	Alta	Validación _user_can_access, ownership	Mitigado

Categoría STRIDE	Amenaza	Componente afectado	Severidad	Contramedida implementada	Estado
	secretos no propios.			y permisos compartidos.	
Elevation of Privilege	Exposición de credenciales CI/CD.	Pipeline	Crítica	Uso de GitHub Secrets para Docker Hub y variables sensibles.	Mitigado

4.5 Riesgos residuales aceptados

Riesgo residual	Justificación	Trabajo futuro
JWT almacenado en cliente en el MVP	Simplifica integración frontend/backend para la entrega académica.	Migrar refresh token a cookie HttpOnly, Secure, SameSite=Strict; access token en memoria.
Sin rate limiting implementado	Alcance MVP centrado en pipeline DevSecOps y arquitectura.	Incorporar slowapi o control equivalente en FastAPI.
TLS no configurado directamente en Docker Compose local	Entorno local usa HTTP; producción debe usar reverse proxy con TLS.	Agregar Nginx/Traefik con certificados TLS.
Monitoring opcional no completamente validado	El requisito lo considera bonificación.	Implementar /metrics real con prometheus-fastapi-instrumentator y dashboards Grafana.
Checkov en modo soft fail	Evita bloquear la entrega por configuraciones aceptadas de entorno académico.	Endurecer reglas y bloquear severidades altas en ramas protegidas.
Trivy SCA en modo warning en el workflow actual	El flujo actual reporta dependencias HIGH/CRITICAL pero no	Cambiar exit-code: 1 para CRITICAL en dependencias.

Riesgo residual	Justificación	Trabajo futuro
	bloquea por dependencias.	

5. Implementación del Pipeline

5.1 Visión general del pipeline

El pipeline DevSecOps se implementa con GitHub Actions y se encuentra en:

```
.github/workflows/ci.yml
.github/workflows/deploy.yml
```

El pipeline CI se ejecuta ante:

```
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]
```

La publicación y liberación se ejecuta solamente cuando el evento corresponde a un push sobre la rama main:

```
if: github.ref == 'refs/heads/main' && github.event_name == 'push'
```

El pipeline CD se activa automáticamente cuando el workflow CI finaliza exitosamente en main:

```
on:
  workflow_run:
    workflows: ["CI – DevSecOps Pipeline"]
    types: [completed]
    branches: [main]
```

5.2 Fase 1 — Plan

La fase de planeación incluye:

- Definición de arquitectura de microservicios.
- DFD nivel 0.
- DFD nivel 1.
- Identificación de activos críticos.
- Modelado STRIDE.
- Archivo threat-model.json para OWASP Threat Dragon.

Artefactos asociados:

```
docs/architecture/threat-model.json
docs/architecture/README.md
```

5.3 Fase 2 — Code

La fase de código integra controles orientados a detectar secretos expuestos, patrones inseguros y vulnerabilidades en dependencias.

5.3.1 Gitleaks — Secret scan

Objetivo:

Detectar secretos hardcodeados en el repositorio y en el historial Git.

Fragmento clave:

```
secrets-scan:
  name: " Secrets Scan (Gitleaks)"
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
      with:
        fetch-depth: 0

    - name: Run Gitleaks
      uses: gitleaks/gitleaks-action@v2
      env:
        GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
```

Interpretación:

- `fetch-depth: 0` permite escanear todo el historial del repositorio.
- Si Gitleaks detecta secretos, el job falla y bloquea la continuidad del pipeline.
- Este control es crítico porque SecureVault gestiona secretos y no debe exponer credenciales en código.

5.3.2 Bandit + Semgrep — SAST

Objetivo:

Detectar patrones inseguros en código Python y reglas OWASP relevantes.

Fragmento clave:

```
sast-scan:
  name: " SAST (Bandit + Semgrep)"
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4

    - name: Set up Python
      uses: actions/setup-python@v5
      with:
        python-version: "3.11"
```

```
- name: Install Bandit
  run: pip install bandit

- name: Run Bandit (Python SAST)
  run: |
    bandit -r api-gateway/app/ -f json -o bandit-report.json -ll || t
rue
    bandit -r worker-audit/ -f json -o bandit-worker-report.json -ll
|| true
```

Fragmento Semgrep:

```
- name: Run Semgrep
  uses: returntocorp/semgrep-action@v1
  with:
    config: >-
      p/python
      p/owasp-top-ten
      p/jwt
      p/secrets
  continue-on-error: true
```

Interpretación:

- Bandit escanea `api-gateway/app/` y `worker-audit/`.
- Semgrep aplica reglas para Python, OWASP Top Ten, JWT y secretos.
- En la versión actual, Bandit y Semgrep están configurados para generar reporte sin bloquear totalmente el pipeline por hallazgos, debido a `|| true` y `continue-on-error: true`.
- Esta decisión debe explicarse como modo de reporte para el MVP, aunque la mejora recomendada es bloquear severidades altas.

5.3.3 Trivy SCA — Dependency scan

Objetivo:

Detectar vulnerabilidades en dependencias de Python y Node.js.

Fragmento clave:

```
dependency-scan:
  name: "Dependency Scan (Trivy SCA)"
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4

    - name: Trivy — API Gateway dependencies
      uses: aquasecurity/trivy-action@master
      with:
        scan-type: fs
```



```
scan-ref: api-gateway/requirements.txt
severity: HIGH,CRITICAL
exit-code: 0
format: table
```

- name: Trivy – Frontend dependencies
uses: aquasecurity/trivy-action@master
with:
 scan-type: fs
 scan-ref: frontend/package.json
 severity: HIGH,CRITICAL
 exit-code: 0
 format: table

Interpretación:

- Se escanean dependencias de backend y frontend.
- En el workflow actual, exit-code: 0 significa que los hallazgos se reportan pero no bloquean.
- Para alineación más estricta con la rúbrica, se recomienda ajustar CRITICAL a exit-code: 1 antes de entrega final.

5.4 Fase 3 — Build

La fase Build construye imágenes Docker de los tres servicios principales y ejecuta escaneo de CVEs sobre las imágenes.

Servicios incluidos en matriz:

```
strategy:
  matrix:
    service:
      - name: api-gateway
        context: ./api-gateway
      - name: worker-audit
        context: ./worker-audit
      - name: frontend
        context: ./frontend
```

Construcción de imagen:

- name: Build Docker image – `${{ matrix.service.name }}`
run: |
 docker build \
 -t securevault/\${{ matrix.service.name }}:\${{ github.sha }} \
 \${{ matrix.service.context }}

Escaneo bloqueante por CVE crítico:

- name: Trivy – Scan image for CVEs
uses: aquasecurity/trivy-action@master

```
with:
  image-ref: securevault/${{ matrix.service.name }}:${{ github.sha }}
  severity: CRITICAL
  exit-code: 1
  format: table
  continue-on-error: false
```

Generación de reporte JSON:

```
- name: Trivy – Full report (HIGH+CRITICAL)
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: securevault/${{ matrix.service.name }}:${{ github.sha }}
    severity: HIGH,CRITICAL
    exit-code: 0
    format: json
    output: trivy-${{ matrix.service.name }}.json
    continue-on-error: true
```

Interpretación:

- Si Trivy encuentra una CVE crítica en una imagen Docker, el pipeline falla.
- Adicionalmente, se genera un reporte JSON para hallazgos HIGH y CRITICAL.
- Este control sí está configurado como bloqueante para severidad CRITICAL en imágenes.

5.5 Fase 4 — Test

La fase de pruebas valida funcionalidad básica de backend y frontend, además de ejecutar análisis dinámico DAST.

5.5.1 Pruebas unitarias backend — Pytest

Fragmento clave:

```
unit-tests-backend:
  name: " Unit Tests – Backend (Pytest)"
  runs-on: ubuntu-latest
  needs: [build-and-scan]
  steps:
    - uses: actions/checkout@v4

    - name: Set up Python 3.11
      uses: actions/setup-python@v5
      with:
        python-version: "3.11"

    - name: Install dependencies
      run: |
        pip install -r api-gateway/requirements.txt
        pip install pytest pytest-cov httpx
```

```
- name: Run Pytest with coverage
env:
  DATABASE_URL: sqlite:///./test.db
  SECRET_KEY: test-secret-key-which-is-long-enough-123456
  FERNET_KEY: TlgmqrngYgDKS7jKdLdZEJn-4RzLMKqy0LqJKJLWqoaA=
  RABBITMQ_URL: amqp://guest:guest@localhost:5672/
run: |
  cd api-gateway
  pytest tests/ --cov=app --cov-report=xml --cov-report=term-missin
g -v
```

Interpretación:

- Las pruebas backend se ejecutan con SQLite en modo test.
- Se genera reporte de cobertura coverage.xml.
- La configuración evita depender de PostgreSQL durante pruebas unitarias.

5.5.2 Pruebas unitarias frontend — Jest

Fragmento clave:

```
unit-tests-frontend:
  name: " Unit Tests – Frontend (Jest)"
  runs-on: ubuntu-latest
  needs: [build-and-scan]
  steps:
    - uses: actions/checkout@v4

    - name: Set up Node 20
      uses: actions/setup-node@v4
      with:
        node-version: "20"

    - name: Install dependencies
      run: cd frontend && npm install

    - name: Run Jest
      run: cd frontend && npm test -- --watchAll=false
```

Interpretación:

- Se validan pruebas unitarias del frontend con Jest.
- Se genera carpeta coverage/ como artifact.

5.5.3 OWASP ZAP — DAST baseline

Fragmento clave:

```
dast-scan:
  name: " DAST (OWASP ZAP)"
```

```
runs-on: ubuntu-latest
needs: [unit-tests-backend, unit-tests-frontend]
steps:
  - uses: actions/checkout@v4

  - name: Start staging environment
    run: |
      docker compose up -d --build
      echo "Waiting for services to be healthy..."
      sleep 30

  - name: Wait for API health
    run: |
      for i in $(seq 1 12); do
        if curl -sf http://localhost:8000/health; then
          echo "API is healthy"
          break
        fi
        echo "Waiting... ($i/12)"
        sleep 5
      done

  - name: OWASP ZAP – Baseline Scan
    uses: zaproxy/action-baseline@v0.12.0
    with:
      target: "http://localhost:8000"
      rules_file_name: ".zap/rules.tsv"
      fail_action: warn
      artifact_name: zap-report
      continue-on-error: true

  - name: Tear down staging
    if: always()
    run: docker compose down -v
```

Interpretación:

- El pipeline levanta un entorno completo con Docker Compose.
- Espera disponibilidad de /health en la API.
- Ejecuta OWASP ZAP Baseline contra http://localhost:8000.
- ZAP se mantiene en modo warn, por lo tanto genera reporte sin bloquear el pipeline.
- Esta configuración es defendible para MVP, ya que ZAP baseline puede generar falsos positivos; sin embargo, hallazgos High deben analizarse y documentarse.

5.6 Fase 5 — Release

La fase Release se ejecuta solo en push a main. Incluye Checkov, login a Docker Hub, publicación de imágenes y creación de GitHub Release.

5.6.1 Checkov — IaC scan

Fragmento clave:

- name: Checkov – IaC Security Scan
uses: bridgecrewio/checkov-action@v12
with:
 directory: infrastructure/terraform
 framework: terraform
 soft_fail: true
 output_format: cli
- name: Checkov – Docker Compose Scan
uses: bridgecrewio/checkov-action@v12
with:
 file: docker-compose.yml
 framework: docker_compose
 soft_fail: true

Interpretación:

- Checkov analiza Terraform y Docker Compose.
- Está configurado en `soft_fail: true`, por lo que reporta sin bloquear.
- Para una política DevSecOps más estricta, se recomienda bloquear hallazgos críticos o documentar excepciones.

5.6.2 Versionamiento y publicación en Docker Hub

Fragmento clave:

- name: Set image tag
id: tag
run: |
 SHORT_SHA=\${GITHUB_SHA:0:8}
 echo "tag=v1.0.0-\${SHORT_SHA}" >> \$GITHUB_OUTPUT
 echo "latest=latest" >> \$GITHUB_OUTPUT

Login Docker Hub:

- name: Login to Docker Hub
uses: docker/login-action@v3
with:
 username: \${ secrets.DOCKERHUB_USERNAME }
 password: \${ secrets.DOCKERHUB_TOKEN }

Build and push:

- name: Build and push all images
run: |
 TAG=\${ steps.tag.outputs.tag }
 REGISTRY=\${ secrets.DOCKERHUB_USERNAME }

```
for service in api-gateway worker-audit frontend; do
  docker build -t $REGISTRY/securevault-$service:$TAG \
    -t $REGISTRY/securevault-$service:latest \
    ./$service
  docker push $REGISTRY/securevault-$service:$TAG
  docker push $REGISTRY/securevault-$service:latest
done
```

Interpretación:

- Se publican tres imágenes:
 - securevault-api-gateway
 - securevault-worker-audit
 - securevault-frontend
- Cada imagen se etiqueta con v1.0.0-<short_sha> y latest.
- Para una interpretación estricta de versionamiento semántico, puede agregarse además un tag estable v1.0.0.

5.6.3 GitHub Release

Fragmento clave:

```
- name: Create GitHub Release
  uses: softprops/action-gh-release@v1
  with:
    tag_name: ${ steps.tag.outputs.tag }
    name: "SecureVault ${ steps.tag.outputs.tag }"
    body: |
      ## Cambios en esta versión
      - Commit: ${ github.sha }
      - Pipeline DevSecOps ejecutado exitosamente
      - Imágenes publicadas en Docker Hub
    draft: false
    prerelease: false
```

5.7 Fase 6 — Deploy

El despliegue se ejecuta desde `.github/workflows/deploy.yml` cuando el pipeline CI finaliza exitosamente en main.

Fragmento clave:

name: CD – Deploy

```
on:
  workflow_run:
    workflows: ["CI – DevSecOps Pipeline"]
    types: [completed]
    branches: [main]
```



```
jobs:
  deploy:
    name: " Deploy with Ansible"
    runs-on: ubuntu-latest
    if: ${{ github.event.workflow_run.conclusion == 'success' }}
    steps:
      - uses: actions/checkout@v4

      - name: Install Ansible
        run: |
          pip install ansible

      - name: Run Ansible deployment
        run: |
          ansible-playbook infrastructure/ansible/site.yml
        env:
          IMAGE_TAG: latest
          DOCKER_REGISTRY: ${{ secrets.DOCKERHUB_USERNAME }}
```

Interpretación:

- El despliegue depende del éxito del pipeline CI.
- Se instala Ansible en el runner.
- Se ejecuta infrastructure/ansible/site.yml.
- El despliegue utiliza variables IMAGE_TAG y DOCKER_REGISTRY.
- Para entorno real, debe configurarse un inventario válido, credenciales SSH y host destino.

5.8 Matriz de controles del pipeline

Control	Herramienta	Fase	Bloquea pipeline	Estado actual
Secret scan	Gitleaks	Code	Sí	Implementado
SAST Python	Bandit	Code	No totalmente	Reporte generado; usa true
SAST reglas OWASP/JWT	Semgrep	Code	No	continue-on-error: true
SCA dependencias	Trivy fs	Code	No	exit-code: 0
Build imágenes	Docker Buildx/Docker build	Build	Sí, si falla build	Implementado

Control	Herramienta	Fase	Bloquea pipeline	Estado actual
CVE crítica en imagen	Trivy image	Build	Sí	exit-code: 1
Reporte HIGH/CRITICAL imagen	Trivy JSON	Build	No	Artifact generado
Pruebas backend	Pytest	Test	Sí	Implementado
Pruebas frontend	Jest	Test	Sí	Implementado
DAST baseline	OWASP ZAP	Test	No	fail_action: warn
IaC scan	Checkov	Release	No	soft_fail: true
Publicación imágenes	Docker Hub	Release	Sí, si falla login/build/push	Implementado
Release	GitHub Release	Release	Sí, si falla acción	Implementado
Deploy	Ansible	Deploy	Sí, si falla playbook	Implementado

6. Resultados de Seguridad

6.1 Gitleaks — detección de secretos

Objetivo del control: detectar API keys, passwords, tokens, claves privadas u otros secretos accidentalmente publicados en el repositorio.

Resultado esperado:

- Si no hay secretos: el job finaliza exitosamente.
- Si hay secretos: el pipeline falla y debe rotarse el secreto expuesto.

En la ejecución del pipeline, Gitleaks realizó el escaneo del repositorio con historial completo (fetch-depth: 0). El objetivo fue identificar secretos hardcoded o credenciales expuestas. El resultado debe analizarse según el artifact o salida del job. En caso de hallazgos, se debe documentar el archivo, tipo de secreto, severidad, acción correctiva y rotación de la credencial.

6.2 Bandit / Semgrep — SAST

Objetivo del control: detectar patrones inseguros en el código fuente, especialmente en Python, autenticación, JWT, OWASP Top Ten y manejo de secretos.

Bandit analizó el código Python ubicado en `api-gateway/app/` y `worker-audit/`. Semgrep complementó el análisis con reglas para Python, OWASP Top Ten, JWT y secretos. Los hallazgos deben clasificarse según severidad y estado: resuelto, mitigado, aceptado o pendiente.

6.3 Trivy — SCA dependencias

Objetivo del control: detectar vulnerabilidades conocidas en dependencias del backend y frontend.

Archivos analizados:

`api-gateway/requirements.txt`
`frontend/package.json`

Trivy ejecutó análisis tipo filesystem sobre archivos de dependencias. En el workflow actual, los hallazgos HIGH/CRITICAL son reportados sin bloquear el pipeline debido a `exit-code: 0`. Se recomienda cambiar a política bloqueante para CRITICAL antes de una entrega productiva.

6.4 Trivy — image scan

Objetivo del control: detectar CVEs en imágenes Docker construidas para los servicios.

Servicios escaneados:

`api-gateway`
`worker-audit`
`frontend`

El pipeline construye tres imágenes Docker en matriz y ejecuta Trivy image scan. La política configurada bloquea el pipeline si se detectan CVEs CRITICAL en imágenes. Adicionalmente, se generan reportes JSON para hallazgos HIGH y CRITICAL.

6.5 OWASP ZAP — DAST

Objetivo del control: realizar análisis dinámico contra la API en ejecución.

Target configurado:

`http://localhost:8000`

OWASP ZAP Baseline se ejecutó contra el entorno levantado con Docker Compose. El job verifica primero `/health` y luego ejecuta el escaneo. En la versión actual, ZAP está en `fail_action: warn`, por lo que genera reporte

sin bloquear el pipeline. Los hallazgos High o Medium deben ser evaluados manualmente y tratados como backlog de seguridad.

6.6 Checkov — IaC scan

Objetivo del control: identificar configuraciones inseguras en Terraform y Docker Compose.

Archivos analizados:

infrastructure/terraform/
docker-compose.yml

Checkov analiza infraestructura como código y configuración de Docker Compose. El workflow actual usa `soft_fail: true`, por lo tanto los hallazgos se reportan sin bloquear. Esta decisión se acepta para el MVP, pero se recomienda endurecer el control en ramas protegidas.

6.7 Tabla de hallazgos de seguridad

ID	Herramienta	Hallazgo	Severidad	Estado	Justificación / Acción
SEC-001	Gitleaks	Sin hallazgos o pendiente de confirmar con ejecución real.	—	Pendiente de evidencia	Insertar resultado real del job. Si hay secretos, rotar credencial y limpiar historial.
SEC-002	Bandit	Posibles advertencias por binds o valores default de desarrollo.	Baja/Media	Aceptado o mitigado	Validar reporte real. En Docker, bind a 0.0.0.0 puede ser necesario dentro del contenedor.
SEC-003	Semgrep	Reglas OWASP/JWT/secrets.	Variable	Pendiente de evidencia	Clasificar hallazgos según severidad real.

ID	Herramienta	Hallazgo	Severidad	Estado	Justificación / Acción
SEC-004	Trivy SCA	Dependencias HIGH/CRITICAL reportadas sin bloqueo en workflow actual.	Alta/Crítica si aplica	Pendiente/Mitigado	Actualizar dependencias vulnerables. Recomendado bloquear CRITICAL.
SEC-005	Trivy Image	CVE CRITICAL en imagen Docker.	Crítica	Resuelto si pipeline pasa	El pipeline bloquea CRITICAL en imágenes con exit-code: 1.
SEC-006	OWASP ZAP	Headers HTTP, CSP, X-Frame-Options u otros hallazgos baseline.	Baja/Media	Aceptado/Mitigado	Revisar reporte real. Agregar headers en Nginx/reverse proxy si aplica.
SEC-007	Checkov	Configuración Docker Compose / Terraform con recomendaciones de hardening.	Baja/Media	Aceptado	soft_fail para MVP; endurecer en producción.
SEC-008	Arquitectura	JWT almacenado en cliente.	Alta	Aceptado	Riesgo residual documentado; migrar a cookies HttpOnly en trabajo futuro.

ID	Herramienta	Hallazgo	Severidad	Estado	Justificación / Acción
SEC-009	Arquitectura	Secretos default en docker-compose.yml para entorno local.	Media	Mitigado parcialmente	Usar .env.example, .gitignore y reemplazar valores antes de producción.
SEC-010	Observabilidad	/metrics puede no estar implementado en API.	Baja	Aceptado	Monitoreo tratado como extensión opcional. Implementar instrumentación real si se reclama bonus.

6.8 Gestión de vulnerabilidades

El proceso de gestión de vulnerabilidades definido para SecureVault es:

1. **Detección:** herramienta del pipeline identifica hallazgo.
2. **Clasificación:** se asigna severidad: Critical, High, Medium, Low o Informational.
3. **Triage:** se valida si es hallazgo real, falso positivo o riesgo aceptado.
4. **Corrección:** se modifica código, dependencia, Dockerfile, workflow o IaC.
5. **Validación:** se reejecuta el pipeline.
6. **Documentación:** se registra estado en tabla de hallazgos.
7. **Excepción justificada:** si no se corrige, se documenta razón, impacto y fecha de revisión.

6.9 Criterio de aceptación de riesgos

Severidad	Política recomendada
Critical	No aceptable sin excepción formal. Debe bloquear build en imágenes.
High	Debe corregirse o justificarse con mitigación.

Severidad	Política recomendada
Medium	Puede aceptarse temporalmente con backlog priorizado.
Low	Documentar y corregir si el esfuerzo es bajo.
Informational	Documentar como mejora.

7. Monitoreo y Observabilidad

7.1 Estado del monitoreo

SecureVault incluye una configuración opcional de observabilidad mediante:

- **Prometheus** para recolección de métricas.
- **Grafana** para visualización.
- **Loki** para almacenamiento centralizado de logs.
- **Promtail** como agente de envío de logs.

Los archivos asociados son:

```
docker-compose.monitoring.yml
monitoring/prometheus/prometheus.yml
monitoring/loki/loki-config.yml
```

El stack se levanta combinando el compose principal y el compose de monitoreo:

```
docker compose -f docker-compose.yml -f docker-compose.monitoring.yml up
-d
```

7.2 Servicios del stack de monitoreo

Servicio	Puerto	Función
Prometheus	9090	Recolección de métricas.
Grafana	3001 host / 3000 container	Dashboards y visualización.
Loki	3100	Almacenamiento de logs centralizados.
Promtail	N/A	Envío de logs hacia Loki.

7.3 Consideraciones técnicas

El archivo `prometheus.yml` incluye targets para:

- Prometheus.
- API Gateway en `/metrics`.
- PostgreSQL exporter.
- RabbitMQ metrics.

Sin embargo, para presentar el monitoreo como completamente funcional se deben validar o implementar los siguientes puntos:

Punto	Estado recomendado antes de entrega
Endpoint /metrics en FastAPI	Implementar con prometheus-fastapi-instrumentator o ajustar configuración.
PostgreSQL exporter	Agregar servicio exporter si se quiere recolectar métricas reales de PostgreSQL.
RabbitMQ metrics	Confirmar plugin/endpoint de métricas en RabbitMQ.
Grafana dashboards	Crear o importar dashboard y capturar evidencia.
Promtail config	Agregar configuración explícita de Promtail si se documenta log shipping real.

8. Conclusiones

8.1 Conclusiones técnicas

SecureVault demuestra la implementación de una aplicación contenerizada con enfoque DevSecOps de ciclo completo. El proyecto integra componentes reales de una arquitectura moderna: frontend SPA, API backend, worker asíncrono, base de datos relacional, broker de mensajería, contenedores Docker, pipeline CI/CD, herramientas de seguridad y automatización de despliegue.

La aplicación permite resolver un caso de uso relevante en ciberseguridad: la administración segura de secretos internos. El uso de cifrado a nivel de aplicación, control de roles, auditoría y rotación de secretos permite conectar la funcionalidad del sistema con objetivos reales de protección de información sensible.

Desde el punto de vista DevSecOps, el mayor valor del proyecto está en el pipeline automatizado. La integración de Gitleaks, Bandit, Semgrep, Trivy, Pytest, Jest, OWASP ZAP y Checkov evidencia una cadena de controles que cubre varias etapas del ciclo de vida del software: código, dependencias, imágenes, pruebas, infraestructura y despliegue.

8.2 Desafíos encontrados

Durante el desarrollo se identificaron varios desafíos técnicos:

1. **Integración entre microservicios:** coordinar la comunicación entre API, PostgreSQL, RabbitMQ y worker requirió healthchecks, dependencias de arranque y retry de conexión.

2. **Persistencia segura de secretos:** fue necesario cifrar los valores antes de almacenarlos, separando el dato sensible de su representación persistida.
3. **Auditoría de acciones sensibles:** se implementó registro de eventos relevantes como creación, revelado, rotación, actualización y eliminación de secretos.
4. **Compatibilidad de pruebas:** el backend se adaptó para ejecutar pruebas con SQLite en modo test y PostgreSQL en ejecución normal.
5. **Pipeline con herramientas múltiples:** integrar herramientas de seguridad diversas generó decisiones sobre qué controles bloquearían el pipeline y cuáles operarían en modo reporte.
6. **DAST en entorno efímero:** OWASP ZAP requiere levantar un entorno funcional antes del escaneo y apagarlo correctamente después.
7. **Publicación de imágenes:** la generación de tags, login a Docker Hub y push de múltiples imágenes requiere configuración correcta de GitHub Secrets.
8. **Documentación y coherencia:** fue necesario alinear diagramas, README, manuales y workflows con el código real para evitar inconsistencias técnicas.

8.3 Limitaciones

Limitación	Impacto	Acción futura
JWT almacenado en cliente en el MVP	Riesgo ante XSS.	Migrar refresh token a cookie HttpOnly y access token en memoria.
Ausencia de rate limiting	Potencial abuso de endpoints.	Implementar slowapi o control equivalente.
TLS no configurado en compose local	HTTP usado en entorno local.	Agregar reverse proxy con TLS para producción.
Checkov en modo soft fail	IaC findings no bloquean release.	Endurecer política por severidad.
Trivy SCA no bloquea dependencias críticas en workflow actual	CVEs de dependencias podrían no bloquear.	Cambiar CRITICAL a exit-code: 1.
Monitoreo opcional no validado completamente	No reclamar bonus sin evidencia.	Implementar /metrics, exporters y dashboards.
Secretos default para demo local	Riesgo si se usan en producción.	Forzar variables obligatorias y usar .env seguro.
Sin MFA	Riesgo si credenciales son comprometidas.	Agregar MFA/TOTP en versión futura.

8.4 Lecciones aprendidas

1. **La seguridad debe diseñarse desde la arquitectura**, no agregarse únicamente al final.

2. **La contenerización facilita reproducibilidad**, pero exige controles sobre imágenes, usuarios, puertos, secretos y redes.
3. **El pipeline DevSecOps debe balancear bloqueo y evidencia**: no todos los hallazgos deben detener el flujo, pero los críticos sí deben estar claramente gobernados.
4. **Los reportes de seguridad son artefactos de entrega**, no simples salidas técnicas; deben interpretarse y documentarse.
5. **La documentación debe reflejar el código real**, especialmente en temas sensibles como hashing, cifrado, almacenamiento de tokens y monitoreo.
6. **La observabilidad requiere instrumentación real**, no solo levantar Prometheus/Grafana.
7. **Los riesgos residuales deben declararse con honestidad**, indicando mitigaciones y trabajo futuro.

8.5 Trabajo futuro propuesto

Mejora	Prioridad	Descripción
Migrar JWT a cookies seguras	Alta	Usar refresh token en cookie HttpOnly, Secure, SameSite=Strict.
Implementar rate limiting	Alta	Proteger login, reveal secret y creación masiva de secretos.
Endurecer pipeline SCA	Alta	Bloquear dependencias con CVE CRITICAL.
Endurecer Checkov	Media	Pasar de soft_fail a bloqueo para hallazgos críticos.
Implementar /metrics real	Media	Instrumentar FastAPI con Prometheus.
Crear dashboards Grafana	Media	Dashboard de API, RabbitMQ, PostgreSQL y Docker.
Agregar MFA/TOTP	Media	Segundo factor para usuarios administradores.
Implementar Vault/KMS externo	Media	Gestionar Fernet key fuera de variables de entorno.
Agregar backup/restore	Media	Copias seguras de PostgreSQL y procedimiento de recuperación.

Mejora	Prioridad	Descripción
Agregar escaneo Kubernetes	Media	Kube-score, kube-linter o Checkov sobre manifests K8s.
Agregar SBOM	Media	Generar SBOM con Syft o Trivy.
Firmar imágenes	Media	Usar Cosign para firmar imágenes Docker.
Implementar políticas OPA	Baja/Media	Validar despliegues con Policy as Code.

8.6 Cierre

SecureVault cumple el objetivo de construir una aplicación funcional y, principalmente, demostrar un pipeline DevSecOps integrado. El proyecto evidencia una visión completa del ciclo de vida seguro del software: desde el modelado de amenazas y diseño de arquitectura, hasta pruebas, escaneos, construcción de imágenes, publicación, despliegue y documentación.

Aunque existen riesgos residuales y oportunidades de mejora, estos han sido identificados y documentados. La entrega representa una base sólida y extensible para evolucionar hacia una plataforma de gestión de secretos más robusta, con mayor endurecimiento de seguridad, observabilidad completa y controles avanzados de operación.